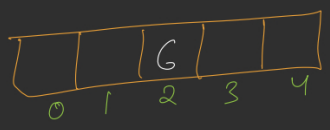
Arrays.

```
int [] arr = new int [5];
```

arr



↳ System.out.println (arr[2]);

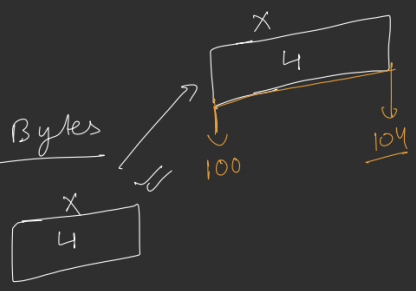
Random Access :

Data Types

- Primitives:
  - ↳ Integer
  - ↳ Floating
  - ↳ Boolean
  - ↳ char
- Non-primitives:
  - ↳ Arrays

int = 4 Bytes

x = 4;



int x = 4;

x = 4 ⊗ 4

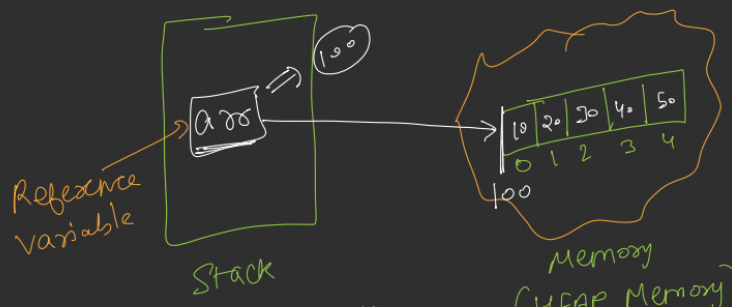
Means 'x' directly hold the Value 4.

⇒ Stack Memory :

Primitive DT

int [] arr = new int [5];

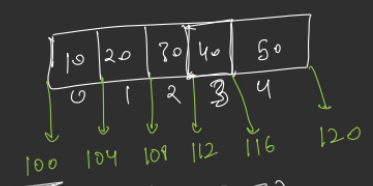
int [] arr = new int [5];



arr it does not directly holds the array.

⇒ It is a reference to an array.

arr[0] = 10 // 100
arr[1] = 20 // 104
arr[2] = 30 // 108
arr[3] = 40;
arr[4] = 50;



System.out.println(arr[3]);

arr[3] = 100 + (4 * 3) = 112

arr[4] = 100 + (4 * 4) = 116

$$\boxed{arr[i] = \text{Base Addr} + (\text{D.T size} * i)}$$

System.out.(arr[3]);

100 + (4 * 3) = 112 ✓

int [] arr = new int[5];

long → 8B | Char → 2B
Float → 4B
Double → 8B

Boolean → Acc. to Official Java Docs
T F
⇓
No bit size

HotSpot (Oracle OpenJDK)
↳ Boolean → 1B ✓

For Better CPU Optimisation

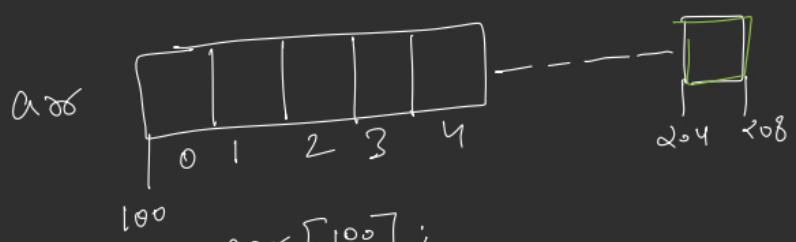
Boolean ⟹ 1B

boolean[] arr = new boolean[6];

↳ 1B

arr[2] = 100 + (1 * 2)

= 102



arr[100];

↳ 100 + (4 * 100)

= 204

ArrayIndexOutOfBound

```
if( index < 0 || index > arr.length) {
    throw ArrayIndexOutOfBound;
}
```

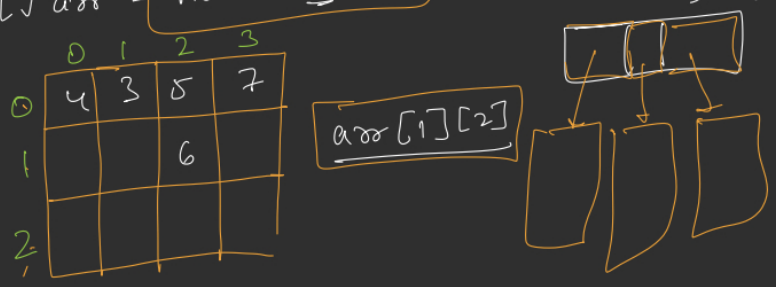
ArrayIndexOutOfBound

```
if( index < 0 || index > arr.length) {
        throw ArrayIndexOutOfBound ;
}
```

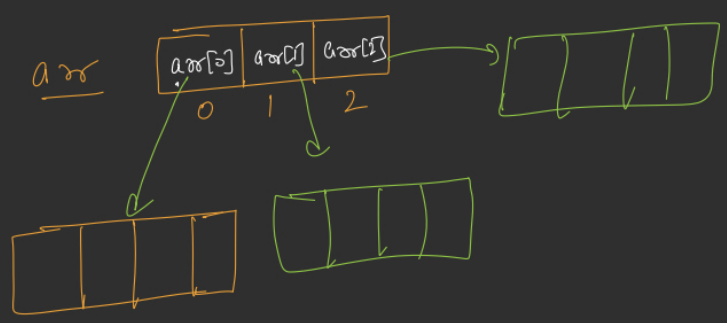
2-D Arrays:

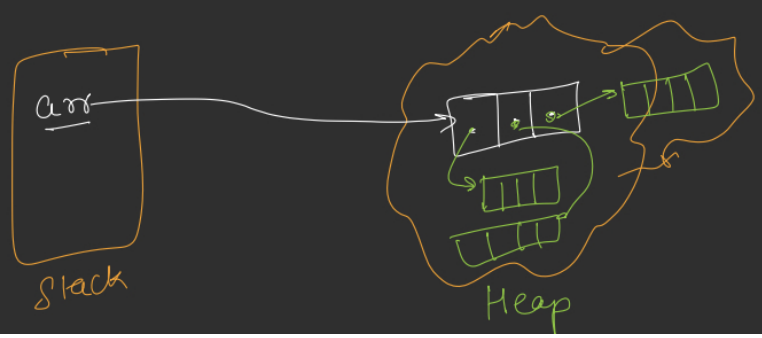
```
int[][] arr = new int[3][4];
```

we've an array of size 3, where each element is itself an array of s=4.

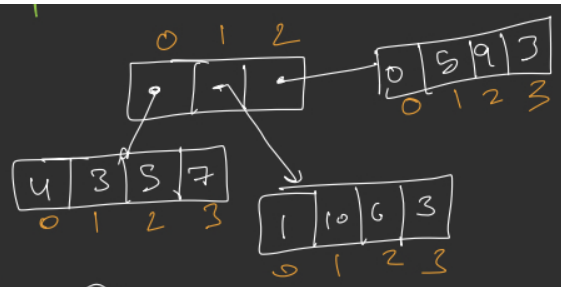


Array of Arrays:





int[][] arr = new int[3][4];



	0	1	2	3
0	4	3	5	7
1	1	10	6	3
2	0	5	9	3

arr[1][2] → 6

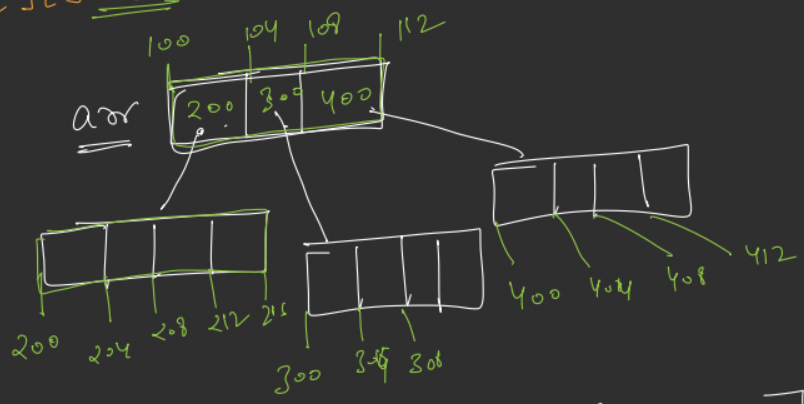
Reference variable → 4B

row → 100 + (4 * 1)

= 104

Col → 200 + (4 * 2) = 208

int[][] arr = new int[3][4];



arr[i][j] → [Base Addr + (4 * i)] = x

Base Addr 2nd Array

row

col → [x + (4 * j)]

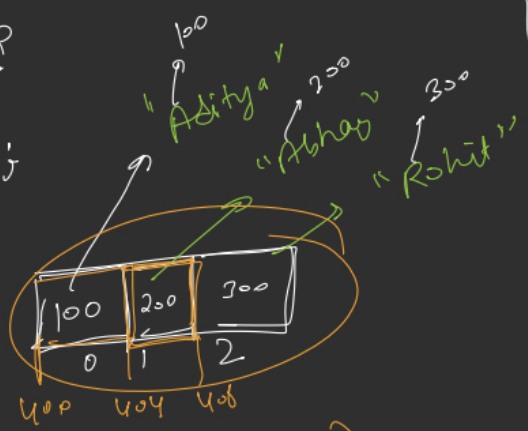
Strings :

String [] names = new String [3];

names → | "Aditya" | "Abhay" | "Rohit" |
0 1 2

String → ??

→ names [1];



= ↳ 400 + (4 * 1)

= 404

char [] name = new char [15];

| A | D | I | T | Y | A | --- |

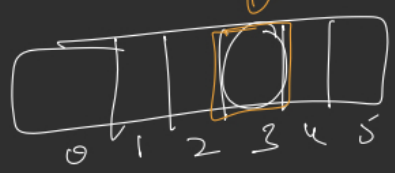
⇓

ADITYA

JDK-9

Random Access

↳ arr [3];



Caching